

SOLID

Five principles that make high cohesion and low coupling repeatable.

From Instinct to Principle

You already know the goal — high cohesion, low coupling, clean code. SOLID gives you five named, teachable moves to get there on purpose.

- SOLID is five object-oriented design principles (Robert C. Martin).
- Each one attacks a specific way that coupling and rigidity creep in.
- They are guidelines, not laws — apply them where change is likely.

The Five Principles

	Principle	In one line
S	Single Responsibility	One class, one reason to change
O	Open / Closed	Open to extension, closed to modification
L	Liskov Substitution	Subtypes must honor the base type's contract
I	Interface Segregation	Many small interfaces beat one fat one
D	Dependency Inversion	Depend on abstractions, not concretions

SOLID Is a Naming Discipline

Every principle is a rule for naming well. A name makes a concept visible — you can point at it — and talkable — the team can discuss it. SOLID is that habit, scaled from the line to the module.

S **SRP** · **Name the responsibility** — *one name, one reason to change*

O **OCP** · **Name the axis of variation** — *variants are names, not a switch*

L **LSP** · **Keep the name's promise** — *a subtype must not make its name a lie*

I **ISP** · **Name each role** — *depend only on the name you use*

D **DIP** · **Depend on the name** — *the abstraction is a named seam*

LSP is the doorway, not the whole room — the name makes a promise; LSP is the rules for keeping it (the behavioral contract).

S — Principle 01

Single Responsibility

A class should have one, and only one, reason to change.

The naming rule

Name the responsibility — *one name, one reason to change*

One Class, One Reason to Change

'Responsibility' means a reason to change. If two different stakeholders can force edits to the same class, it has two responsibilities.

Smells

- A class named ...Manager or ...Utils
- Report + database + formatting in one
- Edits for unrelated reasons collide

Fix

- Separate the reasons to change
- One cohesive job per class
- Compose the jobs from outside

Name the responsibility — no responsibility without a name.

Split the Reasons to Change

Violates

```
class Invoice {  
    double total() {...}  
    void saveToDb() {...}    // persistence  
    String toHtml() {...}   // presentation  
}  
// 3 reasons to change in 1 class
```

Follows

```
class Invoice {  
    double total() {...}  
}  
class InvoiceRepository {    // persistence  
    void save(Invoice i) {...}  
}  
class InvoiceRenderer {      // presentation  
    String toHtml(Invoice i) {...}  
}
```

Now each piece earns a precise name — `InvoiceRepository`, `InvoiceRenderer`: one honest name per job.

O — Principle 02

Open / Closed

Open for extension, closed for modification.

The naming rule

Name the axis of variation — *variants are names, not a switch*

Extend Without Editing

You should be able to add new behavior by adding new code — not by editing code that already works and is already tested.

- Every edit to working code risks breaking it.
- A growing if/switch over 'types' is the classic violation.
- Introduce an abstraction; add new cases as new implementations.

Name the variation — no new behavior without a new name.

The Type-Field Smell

int type + if-else

```
class Employee {  
    int type;    // 1=ENG 2=SALES 3=MGR  
    double pay() {  
        if (type == 1) return base();  
        if (type == 2) return base() + comm();  
        if (type == 3) return base() + bonus();  
        throw new IllegalStateException();  
    }  
}
```

one subclass per role

```
abstract class Employee {  
    abstract double pay();    // its own rule  
}  
class Engineer extends Employee {  
    double pay() { return base(); }  
}  
class Salesman extends Employee {  
    double pay() { return base() + comm(); }  
}  
// Manager, Contractor, ... each a new class
```

‘Replace conditional with polymorphism’ is really naming: `type == 1` becomes a named subclass — Engineer, Salesman. A new role is a new name, not an edited switch.

Add Behavior by Adding Code

Closed to extension

```
double price(Item i) {  
    if (i.type == BOOK) return ...;  
    if (i.type == FOOD) return ...;  
    // new type? edit this method again  
}
```

Open to extension

```
interface Pricing { double price(Item i); }  
class BookPricing implements Pricing {...}  
class FoodPricing implements Pricing {...}  
// new type = new class, no edits here
```

Name the axis of variation — **Pricing** — each variant is a name, not a case.

If it is not SRP, it is not OCP either.

A unit with two reasons to change has no single axis to extend — a new case cuts across both, so you must open the class to add it. Separate the responsibilities first; only then can you extend without editing.

L — Principle 03

Liskov Substitution

Subtypes must be usable anywhere their base type is expected.

The naming rule

Keep the name's promise — *a subtype must not make its name a lie*

Subtypes Must Keep the Promise

If S is a subtype of T, you must be able to use an S wherever a T is expected — without surprises. Inheritance is a promise about behavior.

Violation

- Square extends Rectangle, breaks `setWidth`
- Subtype throws where base didn't
- Subtype weakens the base's guarantees

Test

- Would every caller of the base still work?
- Same inputs → compatible outputs
- If not, it isn't really an 'is-a'

Keep the name's promise — a subtype that breaks it makes the name a lie.

When 'is-a' Lies

Java

Square breaks Rectangle's contract

```
class Rectangle { void setWidth(int w){} void setHeight(int h){} }

class Square extends Rectangle { // a Square 'is-a' Rectangle?
    void setWidth(int w) { super.setWidth(w); super.setHeight(w); }
    void setHeight(int h) { super.setWidth(h); super.setHeight(h); }
}

// A caller that sets width=5, height=4 expects area 20 – gets 16. Broken.
```

Prefer composition over forcing an inheritance that violates the contract.

I — Principle 04

Interface Segregation

No client should depend on methods it does not use.

The naming rule

Name each role — *depend only on the name you use*

Many Small Interfaces

A 'fat' interface forces implementers to support methods they don't need — and couples clients to behavior they never call.

- Split large interfaces along how clients actually use them.
- Implementers provide only what they truly support.
- Clients depend on the small slice they need — nothing more.

ISP is SRP applied to interfaces — and so it brings OCP with it.

One interface, one role, one reason to change: the same rule as SRP, moved from the class to the contract. And once the roles are separated, OCP follows — a new capability is a new interface with a new implementation, and no existing client is edited.

Name each role — depend on the name you use, not the one you don't.

Split the Fat Interface

Fat interface

```
interface Machine {  
    void print();  
    void scan();  
    void fax();  
}  
// a simple printer must fake scan/fax
```

Segregated

```
interface Printer { void print(); }  
interface Scanner { void scan(); }  
interface Fax      { void fax(); }  
// implement only what you support  
class BasicPrinter implements Printer {...}
```

Each interface names a role clients actually use — Printer, Scanner, Fax. Depend on the name you use, nothing more.

D — Principle 05

Dependency Inversion

Depend on abstractions, not on concrete details.

The naming rule

Depend on the name — *the abstraction is a named seam*

Point Dependencies at Abstractions

High-level policy should not depend on low-level details. Both should depend on an abstraction — so details can change without touching policy.

Without DIP

- OrderService creates new MySqlConnection()
- Policy welded to a specific technology
- Can't test without the real database

With DIP

- OrderService depends on a Repository interface
- The concrete DB is injected from outside
- Swap DB or use a fake in tests freely

Depend on the name, not the thing — the abstraction is a named seam.

Inject the Abstraction

Depends on detail

```
class OrderService {  
    MySqlDb db = new MySqlDb();  
    // welded to MySQL forever  
}
```

Depends on abstraction

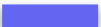
```
class OrderService {  
    final Repository repo;  
    OrderService(Repository r){ repo = r; }  
    // any Repository: MySQL, in-memory, fake  
}
```

This is the engine behind testability and Hexagonal architecture (Phase 2).

 Zoom out

SOLID Is One Idea, Five Ways

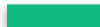
Every principle is a tactic for the same strategy: keep things cohesive and loosely coupled so the system absorbs change.


S · I

Push cohesion up — small,
focused units and interfaces.


O · L · D

Push coupling down —
depend on stable abstractions.


All five

Together: a codebase you can
extend without fear.

SOLID Is Not the Only Set

SOLID shapes a unit once you know what it is for. GRASP answers the question that comes first: which object should own this behavior at all?

GRASP — who should do this?

Nine patterns for assigning responsibility: Information Expert, Creator, Controller, Polymorphism, Pure Fabrication, Indirection, Protected Variations — and the two you already know, Low Coupling and High Cohesion.

Package principles — how should modules group?

Cohesion: REP, CCP, CRP. Coupling: ADP, SDP, SAP. The same thinking as SOLID, one level up — we meet these again in Phase 2.

CUPID — a modern reframing

Dan North's alternative: Composable, Unix philosophy, Predictable, Idiomatic, Domain-based. Properties to aim for rather than rules to obey.

The maxims — short rules that carry a lot

DRY, KISS, YAGNI, Law of Demeter, Tell-Don't-Ask. Not a system, but the fastest checks you own.

 **Applying UML and Patterns, 3rd ed. — Craig Larman · Prentice Hall, 2004** — *the source of GRASP — nine patterns for deciding who does what*

SOLID as a Review Lens

SOLID gives you precise language to steer and critique AI-generated code.

- Ask the AI to flag SRP or DIP violations in a diff before you accept it.
- Name the principle in your prompt: 'make this open/closed over payment types'.
- You judge whether the abstraction earns its keep — don't over-engineer.

The Code Behind This Session

Every example in this session compiles and runs — the same lessons in three languages, with the smell and the fix sitting side by side:

Java

Maven · JUnit 5 · 7 tests here

`solid/srp/ · solid/ocp/`

Invoice split; Employee type-field → subclasses

`solid/lsp/violation/ + fixed/`

Square breaks Rectangle's contract

`solid/isp/ · solid/dip/`

fat interface split; injected Repository

C# / .NET

xUnit · 7 tests here

`Solid.cs`

all five principles, one namespace each

`SolidTests.cs`

one test per principle

the Employee example lives in solid.ocp

Python

pytest · 7 tests here

`solid.py`

all five principles

`tests/test_solid.py`

one test per principle

github.com/kaldiroglu/Agentic-Software-Design-and-Architecture-Bootcamp-JAVA · [-DOTNET](#) · [-PYTHON](#)

LSP is the one to run. The same test passes for Rectangle and fails for Square — the contract breach is visible, not merely argued.

Clone one, run the suite, then break something and watch which test goes red. Across all chapters the suites are 70 · 63 · 65 tests.

Remember these

Key Takeaways

- ✓ SRP: one class, one reason to change.
- ✓ OCP: extend by adding code, not editing working code.
- ✓ LSP: subtypes must honor the base type's contract.
- ✓ ISP: many small interfaces beat one fat one.
- ✓ DIP: depend on abstractions — the key to testability and clean architecture.

Further Reading

The books behind this session. If you read only one, read the first.

📖 **Agile Software Development: Principles, Patterns, and Practices** Robert C. Martin · Prentice Hall, 2002

Where SOLID was first laid out in full, with the case studies behind each principle.

📖 **Clean Architecture** Robert C. Martin · Prentice Hall, 2017

Part III restates the five principles and shows them operating at architectural scale.

📖 **Object-Oriented Software Construction, 2nd ed.** Bertrand Meyer · Prentice Hall, 1997

The origin of the Open/Closed Principle — and of Design by Contract behind LSP.

📖 **Practical Object-Oriented Design, 2nd ed.** Sandi Metz · Addison-Wesley, 2018

SRP and dependency inversion as everyday refactoring moves, not slogans.

📖 **Applying UML and Patterns, 3rd ed.** Craig Larman · Prentice Hall, 2004

The source of GRASP — nine patterns for deciding which object should own a responsibility, the question SOLID assumes you have answered.

Further Reading — Online

Primary sources and online references for this session.

Uncle Bob — the original SOLID principle papers (SRP · OCP · LSP · ISP · DIP)

▶ butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod

Uncle Bob — “Design Principles and Design Patterns” (2000)

▶ objectmentor.com/resources/articles/Principles_and_Patterns.pdf

Liskov & Wing — “A Behavioral Notion of Subtyping” (1994)

▶ dl.acm.org/doi/10.1145/197320.197383

What's Next

Module 5

Design Patterns

with Akin Kaldıroğlu

Factory, Strategy and Proxy — proven solutions that put SOLID to work, so you stop reinventing the same designs.

Turn Principles into Moves

Apply SOLID as concrete refactoring moves on the ACME Orders code — starting with the god-service.

Refactoring course · SOLID

Repo github.com/kaldiroglu/refactoring-course-student-materials

IN THIS SESSION, LOOK AT

- ▶ Exercises/Ex03-ThreeMoves.md , Slides-PDF/Ch03-Principles.pdf
- ▶ .../service/OrderService.java (SRP)
- ▶ .../notify/EmailSender.java , SmsSender.java (DIP / OCP)

Questions?

Five principles. One goal: change without fear.